
PLAYBOOK ACADÉMICO

Manual de Métodos Numéricos

Aplicados a un Simulador de Tráfico 2D en Tiempo Real

UNIDAD I

Newton-Raphson

Sistemas no lineales

UNIDAD III

Simpson 1/3

Integración numérica

UNIDAD IV

Euler Mod. (Heun)

EDO / colas

SOBRE ESTE DOCUMENTO

Guía académica y operativa de los tres métodos numéricos que gobiernan el simulador: su formulación, justificación, algoritmo paso a paso, dónde viven en el código y cómo se combinan para optimizar semáforos y estimar emisiones de CO2.

01 · Arquitectura numérica

El simulador acopla tres unidades de métodos numéricos en un único lazo en tiempo real. La demanda medida por sensores alimenta a Newton-Raphson, que reparte el verde de los semáforos; el método de Heun evoluciona las colas resultantes; y Simpson 1/3 integra esas colas para obtener el retraso y las emisiones de CO2.

FLUJO DE DATOS



RESULTADO

Un reparto de verde adaptado a la demanda que reduce la cola agregada y, por tanto, las emisiones. La comparación plan manual (equitativo) vs. plan optimizado se exporta a Excel con gráficos.

PRINCIPIOS DE DISEÑO

- ▶ Backend numérico puro (numerical_engine.py): solo NumPy, sin dependencias de UI.
- ▶ Implementaciones manuales y explícitas (sin scipy) con fines pedagógicos.
- ▶ Cada método es independiente y reutilizable en otros contextos.

Newton-Raphson (matricial)

FÓRMULA

$$J(X_k) \Delta X = -F(X_k) \Rightarrow X_{k+1} = X_k + \Delta X$$

OBJETIVO EN EL SIMULADOR

Resolver el sistema no lineal del reparto de verde $F(X)=0$: hallar los tiempos de verde $[g_{NS}, g_{EW}]$ que igualan el grado de saturación de ambos ejes según la demanda medida en los sensores.

JUSTIFICACIÓN ACADÉMICA

Convergencia cuadrática cerca de la raíz. Se usa una Jacobiana numérica (diferencias finitas hacia adelante), evitando derivar analíticamente el modelo y manteniendo el método general.

ALGORITMO PASO A PASO

1. Partir de una aproximación inicial X_0 (reparto equitativo).
2. Evaluar el residuo $F(X_k)$ del sistema de equilibrio de flujos.
3. Aproximar la Jacobiana $J(X_k)$ por diferencias finitas.
4. Resolver el sistema lineal $J \cdot \Delta X = -F$ para el paso ΔX .
5. Actualizar $X_{k+1} = X_k + \Delta X$.
6. Repetir hasta que $\|F(X)\| < \text{tol}$ o $\|\Delta X\| < \text{tol}$.

EN EL CÓDIGO

```
numerical_engine.py → solve_green_split(), green_split_residuals()
frontend.py → optimize_traffic_signals() (usa la demanda medida)
```

Regla de Simpson 1/3 compuesta

FÓRMULA

$$\int_a^b q(t) dt \approx \frac{h}{3} [q_0 + 4\sum q_{impar} + 2\sum q_{par} + q_n]$$

OBJETIVO EN EL SIMULADOR

Integrar el área bajo la curva de cola $q(t)$ para obtener las Horas-Vehículo de retraso y, a partir de ellas, la emisión total de CO2 del sistema.

JUSTIFICACIÓN ACADÉMICA

Aproxima la función por parábolas: es exacta para polinomios hasta grado 3, con error $O(h^4)$, superior al trapecio. Si el número de intervalos es impar, se aplica el trapecio solo en el último tramo.

ALGORITMO PASO A PASO

1. Tomar las muestras $q_0..q_n$ de la cola espaciadas un paso h .
2. Si n (intervalos) es impar, reservar el último tramo para trapecio.
3. Ponderar: extremos $\times 1$, nodos impares $\times 4$, nodos pares internos $\times 2$.
4. Multiplicar la suma ponderada por $h/3$.
5. Sumar el trapecio del último tramo si aplica $\rightarrow$ área total.

EN EL CÓDIGO

```
numerical_engine.py  $\rightarrow$  composite_simpson(), integrate_simpson()  
SustainabilityAnalyzer.calculate_total_emissions()  $\rightarrow$  área  $\rightarrow$  CO2
```

Euler Modificado (Heun) · Predictor-Corrector

FÓRMULA

$$\tilde{q}_{i+1} = q_i + h f(t_i, q_i)$$

$$q_{i+1} = q_i + \frac{h}{2} [f(t_i, q_i) + f(t_{i+1}, \tilde{q}_{i+1})]$$

OBJETIVO EN EL SIMULADOR

Evolucionar en el tiempo la longitud de cola $q(t)$ frente al semáforo, resolviendo la EDO $dq/dt = T_{\text{entrada}}(t) - T_{\text{salida}}(t)$ con la restricción física $q(t) \geq 0$.

JUSTIFICACIÓN ACADÉMICA

Como método de segundo orden, su error local es $O(h^3)$ y el global $O(h^2)$: más preciso que Euler simple a igual paso. El corrector promedia la pendiente en los extremos, estabilizando la integración.

ALGORITMO PASO A PASO

1. Evaluar la pendiente inicial $f(t_i, q_i)$.
2. Predictor: estimar $\tilde{q}_{i+1}$ con un paso de Euler.
3. Evaluar la pendiente en el extremo $f(t_{i+1}, \tilde{q}_{i+1})$.
4. Corrector: promediar ambas pendientes para q_{i+1} .
5. Truncar a $q_{i+1} = \max(0, q_{i+1})$ (no hay colas negativas).

EN EL CÓDIGO

```
numerical_engine.py → heun_queue_update(), queue_rhs()  
step() y project_emissions() integran las colas con Heun
```

05 · Guía de uso

CONTROLES

1 - 4

Inyectar vehículos en cada entrada (N-S, E-O, S-N, O-E).

O

Optimizar: mide demanda, resuelve Newton, proyecta y aplica el reparto.

E

Exportar el reporte de sostenibilidad a Excel con gráficos.

H

Ayuda didáctica: overlay con fórmula, objetivo y pasos de cada método.

M

Activar / silenciar la música de fondo.

R

Reiniciar la simulación (nuevo patrón de demanda).

ESC

Salir.

FLUJO RECOMENDADO

1. Deja correr unos segundos: la demanda ambiental es asimétrica (arteria vs. secundaria).
2. Observa las colas crecer en rojo y drenar en verde (método de Heun).
3. Pulsa O para optimizar; el reparto de verde se adapta a la demanda medida.
4. Compara el % de reducción de CO2 en el panel de métricas.
5. Pulsa E para exportar el reporte Excel con el análisis comparativo.

SALIDA: REPORTE EXCEL

El .xlsx incluye Dashboard con KPIs (CO2, retraso, % reducción), desglose por dirección, serie temporal de colas y CO2 acumulado, y equivalentes ambientales (árboles, km, gasolina). Las colas se integran con Simpson 1/3.